T.C.

FATİH SULTAN MEHMET VAKIF ÜNİVERSİTESİ

Mühendislik Fakültesi

Bilgisayar Mühendisliği Bölümü

Lisans Bitirme Projesi

LLM AJANLARI İLE YAPAY ZEKÂ TABANLI ETKİLEŞİMLİ YAZILIM GELİŞTİRME PLATFORMU

Ömer Yasir ÖNAL

2221221562

Bitirme Projesi Danışmanı: Dr. Öğr. Üyesi Nazlı DOĞAN

İstanbul, Kasım 2025

T.C.

FATİH SULTAN MEHMET VAKIF ÜNİVERSİTESİ

Mühendislik Fakültesi

Bilgisayar Mühendisliği Bölümü

Lisans Bitirme Projesi

LLM AJANLARI İLE YAPAY ZEKÂ TABANLI ETKİLEŞİMLİ YAZILIM GELİŞTİRME PLATFORMU

Ömer Yasir ÖNAL

2221221562

Bitirme Projesi Danışmanı: Dr. Öğr. Üyesi Nazlı DOĞAN

Jüri Üyeleri:	İmza:
……..:	.....
……..:	.....
……..:	.....

İstanbul, Kasım 2025

İÇİNDEKİLER

1. PROBLEM TANIMI VE MOTİVASYON 6

1.1. Giriş 6

1.2. Tanımlanan Problemler 6

1.3. Motivasyon ve Katkı 7

1.4. Amaç 7

1.5. Hedefler 8

2. PROJE KAPSAMI 8

2.1. Kapsamın Tanımı 8

2.2. Kapsama Dahil Olanlar 9

A. Platform Çekirdeği ve Temel Özellikler 9

B. Otonom Ajanlar 10

C. Dış Entegrasyonlar 10

D. Akademik ve Teknik Dokümantasyon Çıktıları 10

2.3. Kapsam Dışı Olanlar 11

2.4. Kısıtlar ve Varsayımlar 11

Zaman Kısıtları: 11

Kaynak/Ekip Kısıtları: 12

Maliyet ve Araç Kısıtları: 12

Altyapı Kısıtları: 12

Teknoloji Seçimi Kısıtları: 12

Değişim Yönetimi Kısıtları: 12

Varsayımlar: 12

2.5. Kullanılacak Teknolojiler 13

Çizelge 2.1: Proje teknoloji yığını ve seçim gerekçeleri. 13

2.6. Beklenen Çıktılar 15

3. İLGİLİ ÇALIŞMALAR / LİTERATÜR ARAŞTIRMASI 16
3.1. Araştırma Metodolojisi 16
3.2. Araştırma Anahtar Kelimeleri 16
3.3. Akademik Veri Tabanları ve Kalite Değerlendirmesi 17
3.4. Bulguların Sentezlenmesi 17
3.4.1. Giriş 17
3.4.2. Benzer Yaklaşımlar 18
3.4.3. Karşılaştırmalı Analiz ve AKIS Platformunun Literatüre Katkısı 21
3.5. Literatürdeki Boşluk 22
3.6. Sonuç 23
4. METODOLOJİ VE ÇÖZÜM TEKNİKLERİ 24
4.1. Yaklaşıma Genel Bakış 24
4.2. Yapay Zekâ Ajanı İş Akışı Motoru (AI Agent Workflow Engine) 24
4.2.1. Düzenleyici deseni (Orchestrator Pattern) 24
4.2.2. Ajan fabrikası ve kayıt defteri (Agent Factory and Registry) 25
4.2.3. Ajan durum makinesi (Agent State Machine) 25
4.2.4. Planla-Yürüt-Değerlendir (Planner-Execute-Reflector) pipeline 25
4.2.5. MCP aracı enjeksiyonu 25
4.3. İş Şablonları (Playbooks) ve Sözleşmeler (Contracts) 26
4.3.1. Ajan iş şablonları (Playbooks) 26
4.3.2. Ajan sözleşmeleri (Contracts) 26
4.3.3. Yönetişim ve korkuluklar (Governance and Guardrails) 26
4.4. Geliştirme Metodolojisi 26
4.4.1. Çift hatlı çevik (Dual-Track Agile) 26
4.4.2. Kayan dalga planlaması (Rolling-Wave Planning) 26
4.4.3. Sprint yapısı 27

4.5. Önceliklendirme (WSJF) 27
4.6. Kalite Güvencesi (Quality Assurance) 27
4.6.1. Sola kaydırma testi (Shift-Left Testing) 27
4.6.2. "Bitti" tanımı (Definition of Done - DoD) 27
4.6.3. CI/CD pipeline 28
4.6.4. Hata yönetimi ve gözlemlenebilirlik 28
4.7. Metodolojinin Hedefler (H1–H6) ile Gerekçelendirilmesi 28
5. PROJE YÖNETİMİ 29
5.1. İş Paketleri, Görev Dağılımı ve Zamanlama 29
5.2. Risk Yönetimi 29
6. YAYGIN ETKİ 30
6.1. Ekonomik ve Ticari Etki 30
6.1.1. Geliştirme ekipleri için maliyet azaltma 30
6.1.2. Pazar fırsatı 30
6.1.3. İnovasyon katkısı 30
6.2. Sosyal Etki 31
6.2.1. Geliştirici refahı (Developer Well-Being) 31
6.2.2. Bilgi erişilebilirliği 31
6.2.3. Eğitimsel değer 31
6.3. Sürdürülebilirlik ve Çevresel Etki 31
6.3.1. Kaynak verimliliği 31
6.3.2. Bulut kaynak optimizasyonu 32
6.4. Etik ve Toplumsal Katkı 32
6.4.1. Sorumlu yapay zekâ geliştirme 32
6.4.2. İş gücü etkisi 32
6.4.3. Mühendislik etiği ile uyum 32

KAYNAKLAR 33

EKLER 35

Ek A: İş-Zaman Çizelgesi 35

Ek B: Risk Yönetimi Tablosu 36

1. PROBLEM TANIMI VE MOTİVASYON

1.1. Giriş

Günümüz yazılım geliştirme süreçleri ("Software Development Life Cycle – SDLC") artan kod karmaşıklığı, sürekli değişen gereksinimler ve hızlanan pazara çıkış beklentileri nedeniyle ekipler için önemli operasyonel zorluklar barındırmaktadır. Bu projede, yazılım ekiplerinin verimliliğini düşüren ve maliyetlerini artıran üç ana problem alanı tanımlanmıştır.

1.2. Tanımlanan Problemler

- **Eskimiş Dokümantasyon:** Yazılım projelerinde kod tabanı dinamik olarak değişirken, bu değişikliklerin teknik dokümantasyona (örneğin Confluence) yansıtılması manuel bir süreçtir ve sıklıkla gecikmektedir. Bu durum, dokümanların hızla güncelliğini yitirmesine, bilgi güvenilirliğinin azalmasına ve özellikle ekip içinde yeni katılan geliştiricilerin adaptasyon (onboarding) sürecinin uzamasına yol açmaktadır.
- **Yüksek Test Otomasyon Maliyeti:** Kalite güvencesi (Quality Assurance – QA) süreçlerinde, manuel olarak hazırlanan test senaryolarının otomasyon kodlarına (örneğin Cucumber) dönüştürülmesi yoğun zaman ve efor gerektirir. Bu yüksek maliyet, test kapsamının daralmasına, test süresinin uzamasına ve yazılımın güvenilirliğinin azalmasına neden olmaktadır.
- **Yavaş MVP Prototipleme:** Yeni iş fikirleri veya özelliklerin, en temel çalışan prototipe (Minimum Viable Product – MVP) dönüştürülmesi analiz, tasarım,

kodlama ve test süreçlerinin manuel yürütülmesi nedeniyle günler hatta haftalar sürebilmektedir. Bu yavaşlık, kurumların pazara yenilikleri hızlı sunma ve geri bildirim alma yeteneğini azaltmakta; sonuç olarak rekabet gücü düşmektedir.

1.3. Motivasyon ve Katkı

Bu projenin motivasyonu; yazılım geliştirme süreçlerindeki tekrarlayan, düşük katma değerli görevlerin otomasyona aktarılmasıyla ekiplerin yaratıcı ve stratejik işlere odaklanabilmelerini sağlamaktır.

Akademik ve uygulamalı düzeyde bu çalışma şu katkıları hedefler:

- LLM (Large Language Model) tabanlı otonom ajanların yazılım mühendisliği süreçlerine uygulanması ve bu ajanların “iş şablonları (playbooks)” ile “tasarım/kabul sözleşmeleri (design/acceptance contracts)” biçiminde yapılandırılması.
- Ekiplerin iş akışlarına minimum müdahale ile entegre çalışabilen, modüler ve izlenebilir bir otomasyon platformunun geliştirilmesi.
- Geliştirici verimliliğini artırarak operasyonel maliyetleri düşürmek ve yazılım üretim sürecine “değer ekonomisi” (value economy) bağlamında katkıda bulunmak.

1.4. Amaç

Bu projenin amacı, yazılım geliştirme ekiplerine tekrarlayan görevlerden kurtulma imkânı sağlayarak "zamanı geri kazandırmak" ve ekiplerin yüksek katma değerli işlere odaklanmasına imkân tanımaktır.

Bu doğrultuda, “AKIS Platform” adı verilen ve bir “Yapay Zekâ Ajanı İş Akışı Motoru” (AI Agent Workflow Engine) üzerine kurulu sistem geliştirilerek, dokümantasyon güncelleme, test otomasyonu üretimi ve MVP prototipleme süreçleri otomatikleştirilecektir. Bu sayede hem kalite güvence edilir hem de süreçlerin çevikliği artırılır.

1.5. Hedefler

Bu amacı gerçekleştirmek üzere belirlenen somut, ölçülebilir ve izlenebilir hedefler aşağıda sunulmuştur:

- **H1.** Otonom ajanların görev akışlarını koordine eden ve kullanıcıya web tabanlı bir yönetim arayüzü sunan modüler sistem altyapısını geliştirmek.
- **H2.** Yazılım geliştirme sürecindeki mevcut araçları (GitHub, Jira, Confluence) değiştirmeden entegre şekilde çalışabilen güvenli API bağlantılarını sağlamak.
- **H3.** Teknik dokümantasyonları güncelleyebilen (AKIS Scribe), otomasyon test kodları üretebilen (AKIS Trace) ve verilen gereksinimlerden çalışan prototipler çıkarabilen (AKIS Proto) üç otonom ajanı tasarlayıp hayata geçirmek.
- **H4.** Platformun çıktı kalitesini ölçmek için North Star Metric olarak "kazanılan zaman"ı tanımlamak ve alt metriklerle (Doc Accuracy Score, Diff-Coverage, First-Run Green, Time-to-MVP) izleme altyapısını oluşturmak.
- **H5.** Platformun yüksek güvenlik, kalite güvencesi ve yönetişim standartlarına uygunluğunu sağlamak amacıyla Definition of Done (DoD) kontrol listesi, guard-rails ve hata yönetimi süreçlerini yazılım ve belge düzeyinde uygulamak.
- **H6.** Akademik ve endüstriyel paydaşlara sunulmak üzere, çalışan prototip, kullanıcı dokümantasyonu, demo videosu ve akademik raporu teslim etmek.

2. PROJE KAPSAMI

2.1. Kapsamın Tanımı

AKIS Platformu projesi, yazılım geliştirme yaşam döngüsü (SDLC) süreçlerindeki verimsizlikleri, özellikle dokümantasyon bakımı, test otomasyonu üretimi ve MVP prototipleme görevlerine odaklanarak ele almaktadır. Platform, geliştirme ekiplerinin çabalarını yüksek değerli stratejik çalışmalara yönlendirmelerini sağlamak amacıyla, tekrarlayan ve düşük değerli görevleri otomatikleştiren modüler bir Yapay Zekâ Ajanı İş Akışı Motoru (AI Agent Workflow Engine) olarak tasarlanmıştır.

Kapsam, Oracle Cloud Infrastructure (OCI) Ücretsiz Sürüm kaynakları (4 OCPU, 24 GB RAM) üzerinde dağıtım için optimize edilmiş, hafif ve kaynak açısından verimli bir platformun geliştirilmesini içermektedir. Sistem mimarisi, ölçeklenebilirlik ve sürdürülebilirliği korurken minimum kaynak yükü sağlamak amacıyla otonom ajanları yöneten merkezi bir düzenleyiciye (orchestrator) sahip modüler monolit (modular monolith) bir desen izlemektedir.

Bu kapsam tanımı, neyin, kimin için ve hangi sınırlar dahilinde teslim edileceğini açıklığa kavuşturmaktadır. Ürün kapsamı, metodoloji, teslimatlar, kısıtlar ve başarı kriterlerini içermektedir. Platform, standartlaştırılmış harici entegrasyonlar için Model Context Protocol (MCP) kullanarak, iş akışı değişikliği gerektirmeden mevcut geliştirme araçlarıyla (GitHub, Jira, Confluence) sorunsuz bir şekilde entegre olmaktadır.

2.2. Kapsama Dahil Olanlar

Aşağıdaki maddeler proje kapsamına açıkça dahil edilmiştir:

A. Platform Çekirdeği ve Temel Özellikler

- **A1. Kullanıcı Yönetimi ve Kimlik Doğrulama:** Güvenli kullanıcı oturum yönetimi ve profil yönetimi yetenekleri. Kullanıcılar GitHub OAuth aracılığıyla kimlik doğrulaması yapacak, oturum token'ları Fastify arka ucuna entegre edilmiş hafif bir kimlik doğrulama hizmeti aracılığıyla yönetilecektir.
- **A2. Web Tabanlı Yönetim Arayüzü:** Vite ile oluşturulmuş, React tabanlı bir Tek Sayfa Uygulaması (SPA). Kullanıcılara depo listeleme, ajan görev geçmişini görüntüleme ve yapılandırma yönetimi için bir dashboard sunacaktır.
- **A3. Ajan Görev İzleme Modülü:** Erişilebilir günlük kayıtlarıyla birlikte ajan yürütme durumlarına (beklemede, çalışıyor, tamamlandı, başarısız) gerçek zamanlı görünürlük sağlanacaktır.
- **A4. Ajan Yapılandırma Paneli:** Her ajan türü (Scribe, Trace, Proto) için kullanıcı tarafından yapılandırılabilir parametreler (örn. otomasyon onay ayarları, branch adlandırma stratejileri) sunulacaktır.

B. Otonom Ajanlar

- **B1. AKIS Scribe:** Kod değişikliklerini analiz eden ve ilgili teknik dokümantasyonu (örn. Confluence sayfaları) güncelleyen otonom bir ajan.
- **B2. AKIS Trace:** Jira biletlerinden kabul kriterlerini (acceptance criteria) çıkaran ve bunları yürütülebilir test koduna (örn. Cucumber/Gherkin formatı) dönüştüren otomatik test senaryoları üreten bir ajan.
- **B3. AKIS Proto:** Uçtan uca prototipleme iş akışlarını düzenleyen karmaşık bir ajan. Bir gereksinim veya fikir girdisi verildiğinde, analiz, tasarım, kod üretimi ve test aşamalarını yürüterek çalışan prototipler üretecektir.

C. Dış Entegrasyonlar

- **En1. GitHub Entegrasyonu (MCP ile):** GitHub MCP adaptörü aracılığıyla depolara erişim, commit/PR veri işleme ve dosya işlemleri. Adaptör, doğrudan REST API bağımlılıklarını ortadan kaldırarak MCP istemcisi üzerinden çalışacaktır.
- **En2. Jira Entegrasyonu (MCP ile):** Jira MCP adaptörü aracılığıyla gereksinim ve bilet verilerinin alınması. Bu adaptör, Atlassian'ın Rovo MCP sunucusu ile arayüz oluşturacaktır.
- **En3. Confluence Entegrasyonu (MCP ile):** Confluence MCP adaptörü aracılığıyla dokümantasyon sayfalarının okunması ve güncellenmesi.

D. Akademik ve Teknik Dokümantasyon Çıktıları

- **D1. Proje Kapsamı ve Gereksinimler Dokümanı:** Proje sınırlarını, hedeflerini ve teslimatlarını tanımlayan bu kapsamlı belge.
- **D2. Çözüm Tasarımı ve Teknik Analiz Dokümanı:** OCI Ücretsiz Sürüm kısıtlarıyla uyumlu detaylı mimari dokümantasyonu, API özellikleri ve tasarım gerekçeleri.
- **D3. Final Raporu, Demo Videosu ve Kullanıcı Dokümantasyonu:** Akademik tez raporu, platform yeteneklerini gösteren demo videosu ve son kullanıcı kılavuzları.

2.3. Kapsam Dışı Olanlar

Yönetilebilir kaynak ve zaman kısıtlarını sağlamak amacıyla aşağıdaki maddeler proje kapsamından açıkça çıkarılmıştır:

1. **Kullanıcıların Özel Ajan Geliştirmesi:** Kodsuz/düşük kodlu (no-code/low-code) ajan oluşturma sihirbazları veya kullanıcı tarafından programlanabilir ajan mantığı. Ajanlar önceden tanımlıdır (Scribe, Trace, Proto) ve yalnızca yapılandırılabilir parametrelere sahiptir.
2. **Kurumsal Yönetim Özellikleri:** Temel kullanıcı rolleri dışındaki rol tabanlı erişim kontrolü (RBAC), ekip yönetimi arayüzleri ve şirket içi (on-premise) dağıtım alternatifleri.
3. **Derin Çift Yönlü Entegrasyonlar:** Slack, Microsoft Teams veya VS Code eklentileri ile tam çift yönlü iletişim.
4. **Ücretli LLM Model Optimizasyonu:** Ücretli LLM modelleri veya kaynak yoğun yapay zekâ hizmetleri için sürekli optimizasyon çalışması. Platform, ücretsiz katmanlı yapay zekâ modellerini (örn. OpenRouter aracılığıyla) kullanacaktır.
5. **Ek Ajan Türleri:** Kod inceleme (code review) ajanları, yeniden yapılandırma (refactoring) ajanları veya dağıtım otomasyon ajanları. Kapsam, üç temel ajanla (Scribe, Trace, Proto) sınırlıdır.
6. **Üretim Seviyesi CI/CD Entegrasyonu:** Tam üretim düzeyinde Docker konteynerizasyonu, Kubernetes dağıtımı veya kapsamlı CI/CD pipeline otomasyonu.
7. **Karmaşık SaaS Faturalandırması:** Çok katmanlı abonelik modelleri, kullanıma dayalı faturalandırma veya ödeme ağ geçidi entegrasyonu.

2.4. Kısıtlar ve Varsayımlar

Zaman Kısıtları:

Proje, 26 Ocak'taki demo teslimi ve 22 Mayıs'taki final teslimi gibi kilit kilometre taşlarını içeren akademik takvime sıkı sıkıya bağlıdır.

Kaynak/Ekip Kısıtları:

Geliştirme ekibi iki üyeden (Ömer Yasir ÖNAL, Ayşe Serra GÜMÜŞTAKIM) oluşmaktadır, bu da verimli görev dağılımı ve paralel çalışma akışlarını gerektirmektedir.

Maliyet ve Araç Kısıtları:

Proje, sıfıra yakın bir bütçeyle yürütülmektedir. Tüm bulut hizmetleri, API'ler ve araçlar, ücretsiz katman (free-tier) veya öğrenci lisansı anlaşmaları kapsamında kullanılmalıdır.

Altyapı Kısıtları:

Platform, OCI Ücretsiz Sürüm sınırlamaları (ARM mimarisinde 4 OCPU, 24 GB RAM) dahilinde çalışmalıdır. Bu kısıt, mimari kararları hafif kütüphanelere (Fastify, Drizzle) ve verimli kaynak kullanımına yönlendirmektedir.

Teknoloji Seçimi Kısıtları:

Teknoloji seçimleri, OCI Ücretsiz Sürüm optimizasyon gereksinimleri tarafından yönlendirilmektedir. Mimari; **arka uç için Fastify** (Next.js değil) ve **veritabanı erişimi için Drizzle** (Prisma değil) kullanımını *değiştirilemez kısıt* olarak tanımlar.

Dış entegrasyonlarda **öncelikli strateji** olarak MCP adaptörlerinin kullanılması benimsenmiştir. MCP adaptörleri, doğrudan REST istemcilerine kıyasla varsayılan entegrasyon mekanizmasıdır; ancak gerekçeli durumlarda bu tercih, mimari prensiplerle çelişmemek kaydıyla yeniden değerlendirilebilir.

Değişim Yönetimi Kısıtları:

Proje kapsamı net bir şekilde tanımlanmıştır. Herhangi bir ek özellik, kapsam kaymasını (scope creep) önlemek için resmi bir değişim talebi değerlendirmesi gerektirecektir.

Varsayımlar:

- Kullanıcılar mevcut GitHub, Jira ve Confluence hesaplarına ve uygun izinlere sahiptir.
- Kullanıcılar, Jira/Confluence entegrasyonu için Atlassian API token'larını sağlayabilir.

- Ücretsiz katmanlı yapay zekâ modelleri (OpenRouter aracılığıyla) ajan operasyonları için yeterli kaliteyi sağlamaktadır.
- OCI Ücretsiz Sürüm kaynakları, proje yaşam döngüsü boyunca kullanılabilir olmaya devam edecektir.
- PostgreSQL, uygulama ile aynı sanal makinede (VM) önemli bir performans düşüşü olmaksızın çalışabilecektir.

2.5. Kullanılacak Teknolojiler

Çizelge 2.1: Proje teknoloji yığını ve seçim gerekçeleri.

Kategori	Teknoloji / Araç	Durum	Seçim Gerekçesi	Proje Kısıtları
Çalışma Ortamı	Node.js 20.x	**Zorunlu**	Doğal ARM64 desteği, verimli asenkron G/Ç. Tek süreçli dağıtım yükü azaltır.	4 OCPU ARM VM üzerinde verimli çalışmalı
Arka Uç (Backend)	Fastify + TypeScript	**Zorunlu**	Minimal yük ile yüksek verim (Express'ten ~2 kat hızlı). Eklenti mimarisi modüler ajan tasarımıyla uyumlu.	SSR/CSR yükünü ortadan kaldırmak için Next.js API Routes'un yerini alır
Veri Tabanı	PostgreSQL 15+	**Zorunlu**	Güçlü ilişkisel veritabanı. Ayarlanmış bellek ayarlarıyla (1-2GB) aynı VM'de çalışabilir.	24GB RAM kısıtı içinde çalışmalı
ORM/Query Builder	Drizzle ORM	**Zorunlu**	TypeScript-native, derleme zamanı tip güvenliği. Çalışma zamanı motoru yok (Prisma'nın 6.5MB'ına karşın 1.5MB). Verimli sorgu yürütme.	Bellek ve CPU kullanımını azaltmak için Prisma'nın yerini alır
Ön Uç (Frontend)	React 18+ (Vite SPA)	**Zorunlu**	Modern reaktif UI. Vite, hızlı geliştirme ve optimize edilmiş üretim build'leri sağlar. SPA mimarisi, ön yüzü arka yüzden ayırır.	Statik SPA olarak sunulacak, Next.js arka uçtan çıkarıldı
Stil (Styling)	Tailwind CSS	**Zorunlu**	Utility-first CSS. Minimal çalışma zamanı yükü. Karanlık/Açık tema desteği.	Çalışma zamanında CSS-in-JS yükü yok

Kimlik Doğrulama	GitHub OAuth (Özel)	**Zorunlu**	GitHub entegrasyonu için gerekli. Doğrudan Fastify'da uygulanan hafif OAuth akışı (NextAuth bağımlılığı yok).	GitHub App erişimi ile entegre olmalı
Dış Entegrasyon	MCP Adaptörleri	**Zorunlu**	Yapay zekâ-araç entegrasyonu için standart protokol. GitHub/Jira/Confluence servisleri. Özel REST istemcilerini (Octokit vb.) ortadan kaldırır.	Entegrasyon karmaşıklığını azaltır
Yapay Zekâ Entegr.	OpenRouter API	**Zorunlu**	Tek API üzerinden çoklu ücretsiz LLM modellerine (Llama 3.3, Gemini 2.0 Flash vb.) erişim.	Yalnızca ücretsiz katmanlı AI modellerine izin verilir
Veri Doğrulama	Zod	**Zorunlu**	API girdileri ve ajan yükleri için çalışma zamanı tip doğrulaması.	Ağır çalışma zamanı yükü olmadan tip güvenliği
Loglama	Pino	**Tavsiye Edilen**	Hafif, hızlı JSON logger. Düşük G/Ç yükü.	G/Ç kaynaklarını korumak için verimli loglama
HTTP İstemcisi	Native Fetch (Node.js 20+)	**Zorunlu**	Dahili HTTP istemcisi, ek bağımlılık yok.	Minimum bağımlılık ayak izi
Paket Yöneticisi	pnpm	**Zorunlu**	Verimli disk kullanımı, hızlı kurulumlar, workspace desteği.	node_modules yükünü azaltır
Hosting/Dağıtım	Oracle OCI (Free Tier)	**Zorunlu**	ARM tabanlı VM (4 OCPU, 24 GB RAM). Ücretsiz katman, demo için yeterli kaynak sağlar.	Kaynak kısıtlamaları olan birincil dağıtım hedefi
İzleme	Prometheus + Grafana	**Opsiyonel**	Açık kaynaklı izleme yığını. Minimal yük ile aynı VM'de çalışabilir.	Ücretsiz katmanla uyumlu gözlemlenebilirlik
Dokümantasyon	Markdown, OpenAPI	**Zorunlu**	Teknik dokümantasyon için Markdown. API spesifikasyonu için OpenAPI.	Tüm API'ler belgelenmelidir

2.6. Beklenen Çıktılar

Projenin tamamlanması için aşağıdaki teslimatlar gereklidir:

1. **Çalışan Web Tabanlı Prototip:** Ajan yönetimi, dış entegrasyonlar (GitHub, Jira, Confluence) ve ajan yürütme yetenekleri dahil olmak üzere temel iş akışlarını gösteren işlevsel bir platform. Prototip, OCI Ücretsiz Sürüm altyapısında çalışmalı ve üç ajanı (Scribe, Trace, Proto) uçtan uca göstermelidir.
2. **Üç Temel Ajan için Minimum İşlevsellik:**
    - AKIS Scribe: GitHub depolarında algılanan kod değişikliklerine dayalı olarak Confluence dokümantasyonunu başarıyla güncellemelidir.
    - AKIS Trace: Jira bileti kabul kriterlerinden otomatik test kodu üretmeli ve testleri depoya commit etmelidir.
    - AKIS Proto: Gereksinim girdilerinden kod üretimi, test ve temel dağıtım artefaktları dahil olmak üzere çalışan prototipler üretmelidir.
3. **Demo Sunumu ve Kullanıcı Dokümantasyonu:** Platform yeteneklerini, kullanıcı iş akışlarını sergileyen kayıtlı bir demo videosu. Kurulum kılavuzları ve yapılandırma talimatları dahil olmak üzere kapsamlı kullanıcı dokümantasyonu.
4. **Teknik Dokümantasyon Seti:** Tam sistem mimarisi dokümantasyonu, API spesifikasyonları (OpenAPI/Swagger), ajan sözleşme tanımları ve tasarım gerekçeleri. Dokümantasyon, mimari v2 (Fastify, Drizzle, MCP adaptörleri) ile uyumlu olmalı ve OCI kısıtları bağlamındaki tüm teknoloji seçimlerini gerekçelendirmelidir.
5. **Akademik Tez Raporu:** Problem tanımı, literatür taraması, metodoloji, uygulama detayları, değerlendirme sonuçları ve sonuçları kapsayan resmi akademik belge. Rapor, proje hedefleri (H1–H6) ile elde edilen sonuçlar arasındaki uyumu göstermelidir.

3. İLGİLİ ÇALIŞMALAR / LİTERATÜR ARAŞTIRMASI

3.1. Araştırma Metodolojisi

Bu çalışmada, yazılım mühendisliği süreçlerinde büyük dil modelleri (LLM), otonom ajanlar, çoklu-ajan sistemleri, test otomasyonu, dokümantasyon üretimi ve kod düzeltme gibi alanlarda yayımlanmış güncel araştırmaları incelemek amacıyla sistematik bir literatür taraması yürütülmüştür. Tarama yalnızca hakemli, uluslararası konferans ve dergilerde yayımlanan çalışmalarla sınırlandırılmıştır. Bu kapsamda özellikle bilgisayar mühendisliği ve yapay zekâ alanında A ve A sınıfı olarak kabul edilen ICSE, ASE, MSR, FSE, TSE, NeurIPS, ICLR, ACL ve CHI gibi prestijli organizasyonlarda yayımlanmış çalışmalar tercih edilmiştir.

Literatür taraması, belirlenen anahtar kelimeler çerçevesinde yürütülmüş ve son beş yıl (2020–2025) içinde yayımlanan güncel çalışmalar dikkate alınmıştır. İncelenen yayınlar kullanılan yöntem, çözmeye çalıştıkları problem, elde edilen bulgular ve AKIS platformunun amaçlarıyla ilişkisi açılarak değerlendirilmiştir. Elde edilen bulgular daha sonra tematik gruplar altında sınıflandırılarak analiz edilmiştir.

3.2. Araştırma Anahtar Kelimeleri

Literatür taramasında aşağıdaki anahtar kelimeler kullanılmıştır:

- Autonomous Agents in Software Engineering
  (Otonom ajanların yazılım geliştirme süreçlerinde kullanımı, görev planlama, araç etkileşimi)

- LLM-powered Software Development
  (Büyük dil modellerinin kod üretimi, hata giderme, test ve dokümantasyon süreçlerinde kullanımı)

- Automated Documentation Generation
  (Koddan veya gereksinimlerden otomatik dokümantasyon üretimi)

- AI-assisted Test Generation and Bug Fixing
  (Yapay zekâ ile test senaryosu üretimi ve hata düzeltme süreçleri)

- Multi-Agent LLM Systems
  (Çoklu LLM ajanlarının işbirliği, iletişim protokolleri, görev dağılımı)

- Reasoning Frameworks for LLM Agents
  (ReAct, CoT, ToT gibi akıl yürütme stratejileri, planlama ve araç kullanımı)

Bu anahtar kelimeler, AKIS platformunun üç temel ajanı (Scribe, Proto, Trace) tarafından gerçekleştirilen dokümantasyon üretimi, test otomasyonu ve prototipleme görevlerini açıklayan literatüre erişmek amacıyla seçilmiştir.

3.3. Akademik Veri Tabanları ve Kalite Değerlendirmesi

Literatür taraması kapsamında IEEE Xplore, ACM Digital Library, ScienceDirect, SpringerLink ve Google Scholar veri tabanları kullanılmıştır. Çalışmaların kalitesini değerlendirmek için yalnızca hakemlik sürecinden geçmiş, uluslararası araştırma topluluklarınca tanınan ve bilgisayar bilimlerinde üst düzey kabul edilen konferans ve dergiler seçilmiştir. Bu kapsamda ICSE, ASE, MSR, FSE, NeurIPS, ICLR, ACL ve CHI gibi A/A* seviye konferanslar ile Q1 dergiler öncelikli olarak incelenmiştir.

3.4. Bulguların Sentezlenmesi

3.4.1. Giriş

Yazılım mühendisliği alanında yapay zekâ ve özellikle büyük dil modellerinin (LLM) yükselişi, geleneksel yazılım geliştirme süreçlerini kökten dönüştürmektedir. Kod üretimi, test otomasyonu, dokümantasyon geliştirme ve hata giderme gibi yoğun emek ve tekrar gerektiren faaliyetler, LLM tabanlı otonom ajanlar aracılığıyla giderek daha yüksek düzeyde otomasyonla gerçekleştirilebilmektedir. Son yıllarda yapılan araştırmalar, LLM'lerin karmaşık yazılım mühendisliği görevlerinde bağlam anlayışı, araç kullanımı, planlama ve analiz yetenekleri gösterebildiğini ortaya koymaktadır [1], [2], [3], [4].

Bununla birlikte çoklu ajan etkileşimi, karar verme, prompt mühendisliği, dokümantasyon güncelleme ve yazılım testlerinin doğrulanması gibi alanlarda LLM tabanlı sistemlerin yetkinlikleri hâlâ araştırılmaktadır. Literatürde, otonom ajanların yazılım geliştirme süreçlerine entegrasyonu üzerine önemli çalışmalar yapılmış olsa da, bu çalışmalar belirli işlevlere odaklanmakta ve kapsamlı, uçtan uca çalışan, modüler bir platform yaklaşımını tam olarak sunmamaktadır [1], [4], [5]. Bu nedenle AKIS platformunun hedeflediği Scribe (dokümantasyon), Proto (MVP prototipleme) ve Trace (test otomasyonu) ajanlarının bütünleşik yapısı, mevcut araştırma eğilimleriyle örtüşmekle birlikte literatürdeki boşlukları da tamamlamayı amaçlamaktadır.

Bu bölümde incelenen çalışmalar, tematik olarak üç ana kategori altında sunulmuştur:
(i) Otonom ve çoklu LLM ajanlarının yazılım mühendisliğinde kullanımı,
(ii) LLM tabanlı dokümantasyon üretimi,
(iii) LLM destekli test üretimi, hata giderme ve doğrulama.
Bu tematik sınıflandırma, AKIS platformunun üç çekirdek ajanının görevleriyle doğrudan ilişkilendirilmiştir.

3.4.2. Benzer Yaklaşımlar

3.4.2.1. Otonom ve Çoklu Ajan Sistemleri ile Yazılım Mühendisliği

Son yıllarda LLM tabanlı ajan mimarileri, yazılım geliştirme süreçlerini otomatikleştirmek amacıyla yoğun şekilde araştırılmıştır. “Software Engineering Using Autonomous Agents: Are We There Yet?” çalışması, otonom yazılım ajanlarının mevcut yeteneklerini değerlendirmekte ve yazılım geliştirme görevlerini bağımsız şekilde yerine getirebilme potansiyelini incelemektedir [1]. Çalışmada, LLM destekli ajanların kod yazma, hata tespiti ve planlama gibi görevlerde insan girdisi olmadan hareket edebildiği gösterilmiştir. Bu sonuçlar, AKIS Proto ajanının gereksinim–tasarım–kod–test akışını otomatikleştirme yaklaşımıyla uyumludur.

Benzer şekilde RepairAgent çalışması, otonom bir LLM ajanının hata giderme sürecini planlama, araç çağırma, kod analizi ve çözüm üretme adımlarını içeren bir durum makinesi üzerinden gerçekleştirebildiğini ortaya koymuştur [2]. ICSE düzeyindeki bu çalışma, Proto ajanının araç etkileşimi (tool-use) ve planlama bileşenleri için güçlü bir teorik dayanak sunmaktadır.

Literatürdeki en kapsamlı çoklu ajan çalışması olan “LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead”, çoklu ajanların yazılım geliştirme döngüsündeki rolünü sistematik olarak incelemekte ve görev paylaşımı, işbirliği, koordinasyon ve karar verme süreçlerinin LLM’ler ile nasıl modellenebileceğini açıklamaktadır [3]. Bu çalışma, AKIS platformunun çoklu ajan yaklaşımıyla doğrudan örtüşmektedir.

Buna ek olarak SWE-Agent, NeurIPS 2024’te sunulan bir çalışma olarak, bir LLM ajanının gerçek GitHub depoları üzerinde dosya düzenleme, test çalıştırma, hata tespiti ve düzeltme gibi görevleri otonom olarak gerçekleştirebildiğini göstermiştir [4]. Bu çalışma, AKIS Proto ve Trace ajanlarının test çalıştırma ve kod düzenleme gibi araç tabanlı işlemlerine bilimsel arka plan oluşturmaktadır.

Çok ajan iletişimini temel alan CAMEL çalışması ise, LLM ajanlarının farklı rollere ayrılarak birlikte hedef odaklı çalışabildiğini göstermektedir [5]. Bu çalışma, AKIS platformundaki ajanlar arası etkileşim, görev devri ve çoklu ajan senaryoları için önemli bir referans niteliğindedir.

3.4.2.2. LLM Tabanlı Otomatik Dokümantasyon Üretimi

Otomatik dokümantasyon üretimi, yazılım mühendisliğinde LLM'lerin en hızlı değer yaratan kullanım alanlarından biridir. Khan ve Uddin'in çalışması, GPT-3 tabanlı otomatik kod dokümantasyonu yaklaşımını incelemekte ve modelin fonksiyon seviyesinde tutarlı açıklamalar üretebildiğini göstermektedir [6]. Bu çalışma, AKIS Scribe ajanının kod tabanlı dokümantasyon üretme yeteneği için erken dönem bir temel sunmaktadır.

Dvivedi ve arkadaşları, büyük dil modellerinin (GPT-4, PaLM, LLaMA vb.) dokümantasyon üretimindeki kalite farklarını karşılaştırmış ve farklı modellerin okunabilirlik, doğruluk ve kapsam açısından değişken performans gösterdiğini ortaya koymuştur [7]. Bu sonuçlar, AKIS Scribe’ın model seçimi ve kalite değerlendirme yaklaşımını desteklemektedir.

ACL 2025 Demo Track kapsamında sunulan DocAgent, çoklu ajan yapısında kod dokümantasyonu oluşturmakta ve ajanlar arası görev paylaşımının dokümantasyon kalitesini yükselttiğini göstermektedir [8]. DocAgent, AKIS Scribe’ın çoklu ajan

tasarımına birebir paralel bir sistem olduğundan literatürdeki en yakın çalışma niteliğindedir.

3.4.2.3. LLM Tabanlı Test Üretimi, Hata Giderme ve Doğrulama

LLM tabanlı test otomasyonu son yıllarda önemli bir araştırma alanı hâline gelmiştir. Wang ve arkadaşlarının IEEE TSE'de yayımlanan kapsamlı incelemesi, LLM'lerin test senaryosu oluşturma, sınır durumu keşfetme, hata tahmini ve test veri üretme gibi görevlerde giderek artan doğrulukla kullanılabildiğini göstermektedir [9]. Bu çalışma, AKIS Trace'ın test oluşturma ve doğrulama adımlarının bilimsel temelini oluşturmaktadır.

Bhatia ve arkadaşları ise ICSE 2024 LLM4Code workshop'ında, farklı LLM araçlarının ürettiği birim testlerin performansını karşılaştırarak üretken test araçlarının doğruluk, kapsam ve çalıştırılabilirlik bakımından önemli farklar gösterdiğini ortaya koymuştur [10]. Bu çalışma, Trace ajanının ürettiği testleri değerlendirme süreci için önemli bir referanstır.

LLMSecEval çalışması, doğal dil prompt'larından oluşan güvenlik değerlendirme veri seti sunmakta ve LLM tabanlı araçların güvenlik odaklı davranışlarını anlamayı amaçlamaktadır [11]. Bu çalışma, AKIS platformunda güvenilirlik ve guardrail mekanizmaları açısından önemli bir katkı sağlar.

Son olarak SWT-Bench, LLM tabanlı kod ajanlarının gerçek dünyadaki hata düzeltme işlemlerini test etmek için tasarlanmıştır [12]. Bu çalışma, AKIS Proto'nun hata düzeltme ve doğrulama döngüsüne doğrudan paralelliği nedeniyle literatürdeki en güçlü benzetimli karşılaştırmalardan biridir.

3.4.2.4. LLM Tabanlı Akıl Yürütme Stratejileri

LLM tabanlı ajanların karar verme, görev planlama ve araç kullanma yeteneklerinin temelinde çeşitli akıl yürütme (reasoning) stratejileri yer almaktadır. ReAct (Reason+Act), dil modellerinin düşünme adımlarını (reasoning traces) eylemlerle birleştirerek çevrimiçi karar almasını sağlayan bir yapıdır ve çok adımlı görevlerde modelin bağlamı daha etkili değerlendirmesine olanak tanır [13]. Self-Consistency yaklaşımıyla desteklenen Chain-of-Thought (CoT) tekniği, LLM'lerin karmaşık akıl yürütme görevlerinde daha tutarlı ve güvenilir sonuçlar üretmesini sağlamaktadır [14].

Tree-of-Thoughts (ToT) yöntemi ise, çoklu akıl yürütme yollarını keşfederek problem çözme sürecini dallanma-arama stratejileriyle genişletmekte ve daha yüksek başarı oranı elde etmektedir [15]. Bu yaklaşımlar, AKIS platformundaki ajanların görev planlama, durum analizi ve araç etkileşimi gibi bileşenlerine teorik bir çerçeve sunmakta ve sistemin çok adımlı görevlerde daha kararlı çalışmasına katkıda bulunmaktadır.

3.4.3. Karşılaştırmalı Analiz ve AKIS Platformunun Literatüre Katkısı

İncelenen çalışmalar bütüncül olarak değerlendirildiğinde, LLM tabanlı yazılım mühendisliği araştırmalarının belirli görevlerde dikkate değer başarılar sergilediği görülmektedir. Otonom ajanlar, hata giderme [2], görev planlama [1], çoklu ajan koordinasyonu [5] ve kod düzenleme–doğrulama gibi görevleri gerçek yazılım depoları üzerinde gerçekleştirebilmekte; dokümantasyon üretimi [6], [7], [8] ve test otomasyonu [9], [10], [12] gibi emek yoğun süreçlerde belirgin verimlilik sağlamaktadır.

Ancak mevcut çalışmaların önemli bir bölümü, tekil görev odaklı çözümler sunmakta; yani yalnızca dokümantasyon, yalnızca test üretimi veya yalnızca hata giderme gibi sınırlı problem alanlarına odaklanmaktadır. Örneğin RepairAgent yalnızca hata düzeltme sürecine odaklanırken [2], DocAgent yalnızca dokümantasyon oluşturma görevine yönelmiştir [8]. SWE-Agent ise daha geniş bir etkileşim kapasitesine sahip olsa da [4], belirli senaryolarda tek bir ajan üzerinden yürütülen operasyonel bir yapı sunmaktadır.

Çoklu ajan etkileşimini derinlemesine inceleyen CAMEL [5], rol tabanlı ajan iletişimi için önemli bir çerçeve sunsa da, yazılım mühendisliği özelindeki entegrasyon ve gerçek kurumsal araçlarla çalışma konusunda sınırlıdır. Benzer şekilde SWT-Bench [12] ve LLMSecEval [11] gibi değerlendirme odaklı çalışmalar, belirli görevlerin deneysel analizinde güçlü olmasına rağmen gerçek bir uçtan uca geliştirme süreci sunmaz.

Bu çalışmaların ortak eksikliği, tek başına otonom bir geliştirme ortamı oluşturmamaları, farklı görevleri tek bir platformda birleştirememeleri ve özellikle dokümantasyon–test–prototipleme üçlüsünü bütüncül olarak ele almamalarıdır.

AKIS platformu, bu eksiklikleri gidermeyi hedefleyen üç ajanlı modüler bir mimari sunarak literatüre katkı sağlamaktadır:

- **Scribe ajanı**, kod tabanındaki değişiklikleri okuyup Confluence benzeri dokümantasyon platformlarıyla senkronize ederek literatürdeki otomatik dokümantasyon yaklaşımlarını [6], [7], [8] gerçek kurumsal ortama taşıyan bütünleşik bir çözüm haline getirir.

- **Trace ajanı**, gereksinimlerden otomatik kabul kriterleri çıkartarak [9], [10] gerçek test kodlarını üretir ve çalıştırır; böylece literatürdeki LLM tabanlı test üretim çalışmalarını tam entegre bir test döngüsüne dönüştürür.

- **Proto ajanı**, gereksinim - analiz - tasarım - kod - test sürecini uçtan uca otonom bir pipeline olarak gerçekleştirerek mevcut otonom hata giderme [2] ve gelişmiş agent-tool etkileşimi [4], [12] çalışmalarını daha geniş bir geliştirme yaşam döngüsü bağlamına yerleştirir.

- **Ajanlar arası koordinasyon**, CAMEL'in sağladığı işbirliği prensiplerine [5] dayanarak görev aktarımı, yansıtma (reflection) ve durum makinesi yönetimi gibi yeteneklerle çok daha uygulanabilir ve kurumsal ortama hazır bir yapıya kavuşturulmaktadır.

Dolayısıyla AKIS platformu, literatürdeki tekil çözümlerden farklı olarak dokümantasyon, test, prototipleme, araç entegrasyonu, agent orchestration ve kalite kontrolünü tek bir sistemde birleştirerek çok yönlü bir katkı sunmaktadır.

3.5. Literatürdeki Boşluk

Literatür incelemesi sonucunda şu boşluklar tespit edilmiştir:

1. Uçtan uca yazılım geliştirme sürecini kapsayan çok ajanlı bir LLM sistemi eksiktir. Mevcut çalışmalar ya sadece kod üretimi [4], ya sadece dokümantasyon [6], [8] ya da sadece hata giderme [2] odaklıdır. Tam bir

yazılım yaşam döngüsünü kontrol eden bütünleşik bir mimari sunulmamıştır.

2. Dokümantasyon–test–prototipleme üçlüsünü entegre eden bir platform bulunmamaktadır. Literatürde test otomasyonu [9], [10] ve dokümantasyon [6], [7] ayrı başlıklarda güçlüdür ancak bu araçların birlikte çalıştığı bir yapı yoktur.

3. Kurumsal araçlarla (GitHub, Jira, Confluence) entegre çalışan bir LLM ajan mimarisi eksiktir. Çoğu çalışma laboratuvar ortamı veya sentetik benchmark'lardan oluşmakta, gerçek proje ortamını ele almamaktadır.

4. Ajan yönetimi, workflow kontrolü ve güvenilirlik doğrulaması üzerine tam entegrasyonlu sistemler yok denecek kadar azdır. LLMSecEval [11] ve SWT-Bench [12] gibi güvenilirlik analizleri yalnızca belirli görevleri test etmektedir; tam platform düzeyinde kalite garantisi sunulmamaktadır.

Bu eksiklikler, AKIS'in tasarımını haklı kılmakta ve sunulan platformun literatüre katkısını açıkça göstermektedir.

3.6. Sonuç

Literatür taraması, LLM tabanlı yazılım mühendisliği araçlarının son yıllarda önemli ilerleme kaydettiğini göstermektedir. Dokümantasyon üretimi, test otomasyonu, hata giderme ve çok ajan koordinasyonu üzerine yapılan çalışmalar, LLM'lerin geniş bir görev yelpazesini destekleyebildiğini ortaya koymaktadır. Bununla birlikte bu çalışmalar çoğunlukla belirli görevlerle sınırlı kalmakta ve yazılım yaşam döngüsünün tüm bileşenlerini entegre eden kapsamlı bir platform sunmamaktadır. AKIS platformu, üç ana ajanıyla birlikte dokümantasyon, test ve prototipleme süreçlerini bir araya getiren bütünleşik bir yapı önererek bu boşluğu doldurmayı hedeflemektedir.

4. METODOLOJİ VE ÇÖZÜM TEKNİKLERİ

Bu bölümde, Bölüm 1.5'te tanımlanan hedeflere (H1-H6) ulaşmak için benimsenecek yazılım mühendisliği metodolojileri, mimari yaklaşımlar ve kalite güvence teknikleri detaylandırılmaktadır.

4.1. Yaklaşıma Genel Bakış

AKIS Platformu'nun geliştirme metodolojisi, kaynak kısıtlı bir akademik projeye (OCI Free Tier) ve net mimari kısıtlara (Fastify, Drizzle, MCP) özel olarak uyarlanmış yazılım mühendisliği en iyi uygulamalarını çevik proje yönetimi teknikleriyle birleştirmektedir. Yaklaşım, yinelemeli geliştirmeyi, sola kaydırma (*shift-left*) uygulamalarıyla kalite güvencesini ve proje hedefleriyle (H1–H6) sürekli uyumu vurgulamaktadır.

Metodoloji üç temel direk üzerine inşa edilmiştir:

1. **Modüler Monolit Mimarisi:** OCI Ücretsiz Sürüm'de kaynak verimliliğini en üst düzeye çıkarırken net modül sınırlarını koruyan tek süreçli bir dağıtım modeli.
2. **Ajan Merkezli İş Akışı Motoru:** İyi tanımlanmış sözleşmeler ve iş şablonları (*playbooks*) aracılığıyla otonom ajanları koordine eden merkezi bir düzenleyici (*orchestrator*) deseni.
3. **MCP Tabanlı Entegrasyon Stratejisi:** Harici araç entegrasyonları, Model Context Protocol (MCP) adaptörleri aracılığıyla standartlaştırılarak entegrasyon karmaşıklığı azaltılmış ve platform gelecekteki API değişikliklerine karşı dayanıklı hale getirilmiştir.

4.2. Yapay Zekâ Ajanı İş Akışı Motoru (AI Agent Workflow Engine)

İş Akışı Motoru, platformun otonom ajan yürütmesini sağlayan temel mimari bileşenidir.

4.2.1. Düzenleyici deseni (Orchestrator Pattern)

Merkezi düzenleyici (AgentOrchestrator), ajan yaşam döngüsü yönetiminin tek sahibidir. İş akışı taleplerini alır, bir fabrika deseni (factory pattern) aracılığıyla uygun

ajanları başlatır, her yürütme için durum makinelerini (state machines) yönetir ve araç enjeksiyonunu (MCP adaptörleri, AI hizmetleri) koordine eder. Bu merkezi yaklaşım, kaynak verimliliği (4 OCPU ortamı için kritik olan süreçler arası iletişim yükünü ortadan kaldırır), merkezi durum yönetimi ve yönetişim (governance) sağlar.

4.2.2. Ajan fabrikası ve kayıt defteri (Agent Factory and Registry)

AgentFactory, ajan örneklemesi için bir fabrika deseni uygular. Ajanlar türe göre ("scribe", "trace", "proto") kaydedilir ve talep üzerine bağımlılıkları enjekte edilerek oluşturulur. Bu desen, modüler monolit mimarisiyle uyumlu, eklenti benzeri bir genişletilebilirlik sağlar.

4.2.3. Ajan durum makinesi (Agent State Machine)

Her ajan yürütme işlemi beklemede, çalışıyor, tamamlandı, başarısız durumlarına sahip bir sonlu durum makinesi (FSM) tarafından yönetilir. FSM, geçerli durum geçişlerini zorunlu kılar ve denetim izleri (audit trails) için durum geçişlerini veritabanına kaydeder.

4.2.4. Planla-Yürüt-Değerlendir (Planner-Execute-Reflector) pipeline

Karmaşık ajanlar (özellikle AKIS Proto), üç aşamalı bir yürütme hattı kullanır:

- **Planlama Aşaması:** Ajan, bir Planlayıcı (Planner) AI aracıyla görevi analiz eder ve adımları içeren yapılandırılmış bir plan oluşturur.
- **Yürütme Aşaması:** Ajan, enjekte edilen MCP araçlarını kullanarak ve planı takip ederek birincil görevini gerçekleştirir.
- **Değerlendirme Aşaması:** Bir Değerlendirici (Reflector) AI aracı, yürütme çıktısını (artefakt) eleştirir, sorunları tanımlar ve önerilerde bulunur.

4.2.5. MCP aracı enjeksiyonu

Tüm harici entegrasyonlar, düzenleyici tarafından ajanlara enjekte edilen MCP adaptörleri aracılığıyla sağlanır. Ajanlar, GitHubMCPService, JiraMCPService ve ConfluenceMCPService içeren bir MCPTools nesnesi alır. Bu bağımlılık enjeksiyonu (dependency injection) deseni, ajanları harici servis uygulamalarından ayırır.

4.3. İş Şablonları (Playbooks) ve Sözleşmeler (Contracts)

4.3.1. Ajan iş şablonları (Playbooks)

Bir iş şablonu (AgentPlaybook), bir ajanın yürütme sırasında izlemesi gereken eylem sırasını tanımlar. Strateji (Strategy) ve Komut (Command) desenlerini uygulayarak, adım tanımlarını, yeniden deneme mantığını ve yürütme sırasını kapsüller.

4.3.2. Ajan sözleşmeleri (Contracts)

Ajan sözleşmeleri (AgentContract), ajan yürütmesi için girdi/çıktı şemalarını ve doğrulama kurallarını tanımlar. Çalışma zamanında tip güvenliğini zorlamak için Zod şemalarını kullanır, böylece ajanların geçerli girdiler almasını ve beklenen çıktıları üretmesini sağlar.

4.3.3. Yönetişim ve korkuluklar (Governance and Guardrails)

İş şablonları ve sözleşmeler, merkezi kural zorunluluğu yoluyla yönetişimi mümkün kılar. Düzenleyici, ajan girdilerini sözleşmelere göre doğrular, iş şablonu adımlarını zorunlu kılar ve sözleşme şemalarını ihlal eden ajan çıktılarını reddeder. Bu model, H5 (yüksek güvenlik, kalite güvencesi ve yönetişim standartları) hedefiyle uyumludur.

4.4. Geliştirme Metodolojisi

4.4.1. Çift hatlı çevik (Dual-Track Agile)

Proje, paralel keşif ve teslimat hatlarını sürdüren bir Dual-Track Agile metodolojisini takip eder:

- **Keşif Hattı (Discovery Track):** Mimari tasarım, teknoloji değerlendirmesi ve OCI kısıtlarına uyum üzerine araştırma yapar.
- **Teslimat Hattı (Delivery Track):** Uygulama, test, entegrasyon ve dağıtım yoluyla çalışan yazılımı artımlı olarak üretir.

4.4.2. Kayan dalga planlaması (Rolling-Wave Planning)

Detaylı planlama sadece yakın vadeli işler (mevcut ve bir sonraki sprint) için yapılırken, üst düzey kilometre taşları uzun vadeli yönlendirmeyi sağlar. Bu, akademik araştırma projelerindeki belirsizliği kabul eder ve adaptasyona izin verir.

4.4.3. Sprint yapısı

Sprintler, iki haftalık yinelemelerdir ve Sprint Planlama, Günlük Toplantılar (Daily Standups), Sprint Gözden Geçirme (Review) ve Sprint Retrospektifi (Retrospective) içerir.

4.5. Önceliklendirme (WSJF)

İş önceliklendirmesi, Ağırlıklı En Kısa İşe Öncelik (Weighted Shortest Job First - WSJF) yöntemini kullanır. Bu, yüksek değerli, kısa süreli işleri önceliklendirerek değer sunumunu maksimize eder.

WSJF Formülü: WSJF = (Kullanıcı-İş Değeri + Zaman Kritikliği + Risk Azaltma) / İş Süresi

Bu yaklaşım, proje hedeflerini (H1–H6) doğrudan ele alan veya entegrasyon riskini (örn. MCP adaptörü) azaltan işlerin önceliklendirilmesini sağlar.

4.6. Kalite Güvencesi (Quality Assurance)

4.6.1. Sola kaydırma testi (Shift-Left Testing)

Kalite güvencesi, geliştirme yaşam döngüsünün başlarına entegre edilmiştir:

- **Birim Testleri (Unit Testing):** Ajan mantığı, düzenleyici fonksiyonları ve MCP adaptörleri izolasyon içinde test edilir.
- **Sözleşme Testleri (Contract Testing):** Ajan sözleşmeleri, şema tabanlı testlerle doğrulanır.
- **Entegrasyon Testleri:** MCP adaptör entegrasyon testleri, protokol uyumluluğunu doğrular (mock MCP sunucuları kullanılarak).
- **Uçtan Uca (E2E) Testler:** Kritik kullanıcı iş akışları, API seviyesinde E2E testleri aracılığıyla test edilir.

4.6.2. "Bitti" tanımı (Definition of Done - DoD)

Her iş öğesi, kabul edilmeden önce DoD kontrol listesini karşılamalıdır:

- Kod, tüm linting ve tip kontrollerinden geçer.
- Tüm testler geçer.

- Kod başarıyla build edilir.
- Mimari uyumluluk doğrulandı (Fastify, Drizzle, MCP - yasaklı teknolojiler yok).
- Dokümantasyon güncellendi (API dokümanları, ajan sözleşmeleri).
- Güvenlik hususları ele alındı (kimlik doğrulama, girdi doğrulaması).

4.6.3. CI/CD pipeline

Sürekli Entegrasyon (CI), GitHub Actions iş akışları aracılığıyla uygulanır: Tip Kontrolü, Linting, Test, Build Doğrulaması ve Mimari Uyumluluk kontrollerini içerir.

4.6.4. Hata yönetimi ve gözlemlenebilirlik

Hata yönetimi, yapılandırılmış hata yanıtları ve Pino ile yapılandırılmış loglama kullanır. Gözlemlenebilirlik; Prometheus metrikleri, loglar ve sağlık kontrolü (health endpoints) uç noktaları (/health, /ready) aracılığıyla sağlanır. Bu altyapı, H4 (izleme ve metrik altyapısı) hedefini destekler.

4.7. Metodolojinin Hedefler (H1–H6) ile Gerekçelendirilmesi

- **H1 (Modüler Altyapı):** Modüler monolit mimarisi ve düzenleyici deseni, modüler bir çekirdeği doğrudan destekler.
- **H2 (Güvenli API Bağlantıları):** MCP adaptör deseni ve bağımlılık enjeksiyonu metodolojisi, güvenli ve standartlaştırılmış entegrasyonlar sağlar. Sola kaydırma testleri, MCP adaptörlerinin güvenilirliğini garanti eder.
- **H3 (Üç Otonom Ajan):** Ajan merkezli iş akışı motoru metodolojisi, ajan geliştirme için tutarlı bir çerçeve sunar. Planla-Yürüt-Değerlendir pipeline'ı, AKIS Proto'nun karmaşık gereksinimlerini özel olarak destekler.
- **H4 (İzleme ve Metrikler):** Sola kaydırma gözlemlenebilirlik uygulamaları (metrikler, loglama) baştan entegre edilmiştir.
- **H5 (Güvenlik ve Yönetişim):** "Bitti" Tanımı (DoD), sözleşme tabanlı doğrulama ve CI/CD pipeline'ı sistematik kalite kontrolleri sağlar. Düzenleyicinin yönetişim yetenekleri, ajanların kalite standartlarına uymasını sağlar.

- **H6 (Prototip ve Dokümantasyon):** Dual-Track Agile, dokümantasyon (akademik rapor, kullanıcı kılavuzları) ve uygulama çalışmalarının paralel ilerlemesini sağlar.

5. PROJE YÖNETİMİ

5.1. İş Paketleri, Görev Dağılımı ve Zamanlama

Proje, iki akademik dönemi kapsayan 10 aylık bir süreçte tamamlanacaktır. Geliştirme süreci, İş Kırılım Yapısına (WBS) uygun olarak 10 ana İş Paketine (İP) bölünmüştür. Bu iş paketleri arasında proje tanımlama (İP1), platform çekirdeği geliştirme (İP2), MCP entegrasyonları (İP3) ve ajan geliştirmeleri (İP5, İP7) gibi kritik görevler bulunmaktadır.

Her iş paketinin sorumlu kişileri ve aylara göre dağılımını gösteren detaylı **İş-Zaman Çizelgesi**, belgenin sonunda **Ek A**'da sunulmuştur.

5.2. Risk Yönetimi

Proje yönetimi sürecinin temel bir parçası olarak, başarıyı olumsuz etkileyebilecek potansiyel teknik ve idari riskler proaktif olarak tanımlanmıştır. Başlıca riskler arasında MCP entegrasyon karmaşıklığı (R1), OCI altyapı performans kısıtları (R2), yapay zekâ model kalitesi (R3) ve potansiyel kapsam kayması (R4) bulunmaktadır.

Her risk için tanımlanan B Planı ve azaltma tedbirlerini içeren detaylı **Risk Yönetimi Tablosu**, belgenin sonunda **Ek B**'de sunulmuştur.

6. YAYGIN ETKİ

6.1. Ekonomik ve Ticari Etki

6.1.1. Geliştirme ekipleri için maliyet azaltma

AKIS Platformu, yazılım geliştirmedeki üç yüksek maliyetli verimsizliği doğrudan ele almaktadır:

- **Dokümantasyon Bakım Maliyetleri:** Manuel dokümantasyon güncellemeleri, tipik ekiplerde geliştirme zamanının %5-10'unu tüketmektedir. AKIS Scribe, bu süreci otomatikleştirmeyi hedefler.
- **Test Otomasyon Geliştirme Maliyetleri:** Manuel test senaryolarını otomatikleştirilmiş test kodlarına dönüştürmek maliyetlidir. AKIS Trace, bu dönüşümü otomatikleştirerek test geliştirme süresini %60-80 oranında azaltmayı ve böylece sürüm döngülerini hızlandırmayı amaçlar.
- **MVP Prototipleme Maliyetleri:** Geleneksel MVP geliştirmesi haftalar sürebilir. AKIS Proto, bunu saatlere veya günlere indirerek iş fikirlerinin hızla doğrulanmasını ve pazara daha hızlı çıkılmasını (*faster time-to-market*) sağlar.

6.1.2. Pazar fırsatı

Yazılım geliştirme araçları pazarı, yapay zekâ destekli otomasyona yönelik artan bir taleple büyümektedir. AKIS Platformu, SDLC otomasyonu nişine odaklanarak net bir farklılaşma sunar: mevcut araçlarla (GitHub, Jira, Confluence) sorunsuz entegrasyon ve kaynak verimliliği (OCI Ücretsiz Sürüm dağıtımı).

6.1.3. İnovasyon katkısı

Proje, düşük değerli, tekrarlayan görevleri otomatikleştirerek yazılım geliştirmede "değer ekonomisine" katkıda bulunur. Bu, ekiplerin zaman yoğun manuel çalışmalardan, yapay zekâ destekli otomatize iş akışlarına geçişini temsil etmektedir.

6.2. Sosyal Etki

6.2.1. Geliştirici refahı (Developer Well-Being)

Tekrarlayan görevlerin otomasyonu, geliştirici tükenmişliğini (*burnout*) azaltır ve iş tatminini artırır. Geliştiriciler, sıkıcı dokümantasyon güncellemeleri yerine yaratıcı problem çözmeye odaklanabilirler.

6.2.2. Bilgi erişilebilirliği

Otomatik dokümantasyon güncellemeleri, teknik bilginin güncel ve erişilebilir kalmasını sağlar. Bu, özellikle dağıtık ekipler ve açık kaynaklı projeler için değerli olan yeni ekip üyelerinin adaptasyon (*onboarding*) süresini kısaltır.

6.2.3. Eğitimsel değer

AKIS Platformu, yazılım mühendisliği öğrencileri ve genç geliştiriciler için modern yazılım mimarisi desenlerini (modüler monolit, düzenleyici), yapay zekâ entegrasyonunu ve kaynak kısıtlı sistem tasarımını (OCI optimizasyonu) gösteren bir eğitim aracı olarak hizmet vermektedir.

6.3. Sürdürülebilirlik ve Çevresel Etki

6.3.1. Kaynak verimliliği

AKIS Platformu'nun mimarisi, OCI Ücretsiz Sürüm (4 OCPU, 24 GB RAM) üzerinde çalışmak üzere açıkça kaynak verimliliği için optimize edilmiştir. Bu verimlilik, kaynak yoğun alternatiflere kıyasla enerji tüketimini azaltır:

- **Hafif Teknoloji Yığını:** Fastify ve Drizzle kullanımı, bellek ayak izini ve sorgu CPU kullanımını önemli ölçüde azaltır.
- **Tek Süreçli Dağıtım:** Modüler monolit (mikroservislere karşı), çoklu çalışma zamanı yüklerini ortadan kaldırır.

Bu tek süreçli tasarım, çoklu VM'lere ihtiyaç duyan mikroservis dağıtımlarına kıyasla enerji tüketiminde %60-70'e varan bir azalma potansiyeli sunar.

6.3.2. Bulut kaynak optimizasyonu

Platformun "minimum kaynakla maksimum verimlilik" tasarım felsefesi, sürdürülebilir bulut bilişim uygulamalarını teşvik eder. Karmaşık yapay zekâ sistemlerinin ücretsiz katman kaynaklarında verimli bir şekilde çalışabileceğini göstererek, sektördeki aşırı kaynak sağlama (*over-provisioning*) eğilimine meydan okur.

6.4. Etik ve Toplumsal Katkı

6.4.1. Sorumlu yapay zekâ geliştirme

AKIS Platformu, sorumlu yapay zekâ uygulamalarını şu yollarla hayata geçirir:

- **Şeffaflık:** Ajan iş şablonları (*playbooks*) ve sözleşmeleri, yapay zekâ karar alma süreçlerinde görünürlük sağlar.
- **Yönetişim:** Merkezi kural zorunluluğu, yapay zekâ ajanlarının kalite ve güvenlik standartlarına uymasını sağlar.
- **Denetlenebilirlik:** Yürütme günlükleri, planlar ve eleştiriler saklanarak yapay zekâ davranışının sonradan analiz edilmesine olanak tanır.
- **İnsan Gözetimi:** Kullanıcılar ajan parametrelerini yapılandırır ve otomasyonu onaylar, böylece insan kontrolü korunur.

6.4.2. İş gücü etkisi

Platformun felsefesi, insanın yerini almaktan ziyade onu güçlendirmeye (*augmentation over replacement*) odaklanır. Düşük değerli görevleri otomatikleştirerek, geliştiricilerin insan muhakemesi ve yaratıcılık gerektiren (mimari tasarım, stratejik karar alma gibi) yüksek değerli işlere odaklanmasını sağlar.

6.4.3. Mühendislik etiği ile uyum

Proje, sistematik kalite kontrolleri (DoD, test), şeffaflık (ajan yeteneklerinin net dokümantasyonu) ve kullanıcı özerkliği (yapılandırma ve onay mekanizmaları) sağlayarak yazılım mühendisliği etiğiyle uyum sağlar.

KAYNAKLAR

[1] Suri, S., Das, S. N., Singi, K., Dey, K., Sharma, V. S., & Kaulgud, V. (2023, September). Software engineering using autonomous agents: Are we there yet?. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (pp. 1855-1857). IEEE.

[2] Bouzenia, I., Devanbu, P., & Pradel, M. (2025). *RepairAgent: An autonomous, LLM-based agent for program repair*. In IEEE/ACM 47th International Conference on Software Engineering (ICSE '25) (pp. 2188–2200). IEEE Press. https://doi.org/10.1109/ICSE55347.2025.00157

[3] He, J., Treude, C., & Lo, D. (2025). *LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead*. ACM Transactions on Software Engineering and Methodology, 34(5), Article 124. https://doi.org/10.1145/3712003

[4] Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). *SWE-agent: Agent-computer interfaces enable automated software engineering*. In *Advances in Neural Information Processing Systems* (NeurIPS 2024). https://doi.org/10.52202/079017-1601

[5] Li, G., Hammoud, H. A. K., Itani, H., Khizbullin, D., & Ghanem, B. (2023). *CAMEL: Communicative agents for "mind" exploration of large language model society*. In *Advances in Neural Information Processing Systems* (NeurIPS 2023).

[6] Khan, J. Y., & Uddin, G. (2023). *Automatic code documentation generation using GPT-3*. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)* (Article 174, pp. 1–6). Association for Computing Machinery. https://doi.org/10.1145/3551349.3559548

[7] Dvivedi, S. S., Vijay, V., Pujari, S. L. R., Lodh, S., & Kumar, D. (2024). *A comparative analysis of large language models for code documentation generation*. In *Proceedings of the 1st ACM International Conference on AI-Powered Software (AIware 2024)* (pp. 65–73). Association for Computing Machinery. https://doi.org/10.1145/3664646.3664765

[8] Yang, D., Simoulin, A., Qian, X., Liu, X., Cao, Y., Teng, Z., & Yang, G. (2025). *DocAgent: A multi-agent system for automated code documentation generation*. In

Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations) (pp. 460–471). Association for Computational Linguistics.

[9] Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., & Wang, Q. (2024). *Software testing with large language models: Survey, landscape, and vision*. IEEE Transactions on Software Engineering, 50(4), 911–936. https://doi.org/10.1109/TSE.2024.3368208

[10] Bhatia, S., Gandhi, T., Kumar, D., & Jalote, P. (2024). *Unit test generation using generative AI: A comparative performance analysis of autogeneration tools*. In *Proceedings of the 1st International Workshop on Large Language Models for Code (LLM4Code '24)* (pp. 54–61). Association for Computing Machinery. https://doi.org/10.1145/3643795.3648396

[11] Tony, C., Mutas, M., Díaz Ferreyra, N. E., & Scandariato, R. (2023). *LLMSecEval: A dataset of natural language prompts for security evaluations*. In *Proceedings of the 2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR '23)* (pp. 588–592). IEEE. https://doi.org/10.1109/MSR59073.2023.00084

[12] Mündler, N., Müller, M. N., He, J., & Vechev, M. (2024). *SWT-Bench: Testing and validating real-world bug-fixes with code agents*. In *Advances in Neural Information Processing Systems* (NeurIPS 2024). https://doi.org/10.52202/079017-2601

[13] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing reasoning and acting in language models*. In *Proceedings of the International Conference on Learning Representations (ICLR 2023)*.

[14] Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., & Zhou, D. (2023). *Self-consistency improves chain-of-thought reasoning in language models*. In *Proceedings of the International Conference on Learning Representations (ICLR 2023)*.

[15] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023). *Tree of thoughts: Deliberate problem solving with large language models*. In *Advances in Neural Information Processing Systems* (NeurIPS 2023).

EKLER

Ek A: İş-Zaman Çizelgesi

İş Paketi			Zaman (Aylar)									
No	**Adı**	**Sorumlu kişi(ler)**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
İP1	Proje Başlangıcı ve Tanımlama (Proje Öneri Formu, Kapsam ve Gereksinimler Dokümanı)	Ömer Yasir ÖNAL, Ayşe Serra GÜMÜŞTAKIM	X	X	X							
İP2	Platform Çekirdeği Geliştirme (Fastify, Drizzle, PostgreSQL Mimarisi)	Ömer Yasir ÖNAL	X	X	X							
İP3	MCP Entegrasyonları (GitHub, Jira, Confluence Adaptörleri)	Ömer Yasir ÖNAL		X	X	X						
İP4	Ön Uç (Frontend) Geliştirme (React SPA, Dashboard, Kimlik Doğrulama)	Ayşe Serra GÜMÜŞTAKIM		X	X	X						
İP5	Temel Ajan Geliştirme (AKIS Scribe ve AKIS Trace)	Ömer Yasir ÖNAL			X	X	X					
İP6	Dönem I Teslimatları (Çözüm Tasarımı Dokümanı ve Dönem Sonu Demosu)	Ömer Yasir ÖNAL, Ayşe Serra GÜMÜŞTAKIM				X						
İP7	Gelişmiş Ajan Geliştirme (AKIS Proto - MVP Prototipleme)	Ömer Yasir ÖNAL						X	X	X		
İP8	Ön Uç Arayüz İyileştirmeleri (Ajan Yönetim Panelleri, Loglama UI)	Ayşe Serra GÜMÜŞTAKIM						X	X			
İP9	Uçtan Uca Testler, OCI Dağıtımı ve Performans Optimizasyonu	Ömer Yasir ÖNAL, Ayşe Serra GÜMÜŞTAKIM								X	X	
İP10	Dönem II Teslimatları (Final Raporu, Poster Sunumu ve Proje Kapanışı)	Ömer Yasir ÖNAL, Ayşe Serra GÜMÜŞTAKIM									X	X

Ek B: Risk Yönetimi Tablosu

İP No	Risk(ler)in Tanımı	Alınacak Tedbir (ler) (B Planı)
İP3	MCP Entegrasyon Karmaşıklığı ve Belirsizliği: Projenin (H2) hedefinin temel taşı olan Model Context Protocol (MCP) adaptörlerinin geliştirilmesi. Bu protokolün yeni olması, dokümantasyonunun sınırlı olması ve Atlassian (Rovo) veya GitHub MCP sunucularında yaşanabilecek beklenmedik API değişiklikleri veya protokol uyumsuzlukları nedeniyle, entegrasyon sürecinin tahmin edilen eforu (İP3 zaman çizelgesini) aşması riski.	1. Erken Kavram Kanıtı (PoC): Projenin ilk fazında (Ay 1-2) derhal bir "proof-of-concept" çalışması yürütülecektir. Bu, en riskli varsayım olan MCP protokolünün uygulanabilirliğini erkenden doğrulamak için yapılacaktır. 2. Sahte (Mock) Sunucu Geliştirme: Geliştirme ve test süreçlerini harici API'lerin kararlılığına bağımlı kılmamak için, test edilebilir bir sahte (mock) MCP sunucusu oluşturulacaktır. 3. Acil Durum Planı (Sınırlı Fallback): MCP entegrasyonunun (Ay 2 sonuna kadar) teknik olarak imkânsız veya beklenenden çok daha maliyetli olduğu kanıtlanırsa, *sadece entegrasyon katmanında* sınırlı bir fallback uygulanacaktır. Bu senaryoda, aynı adapter arayüzünü kullanan REST tabanlı adapter'lar (ör. Octokit, Jira REST API) devreye alınacak, üst katmanlar MCP benzeri soyutlama ile çalışmaya devam edecektir. Bu fallback, Bölüm 2.4'te tanımlanan "MCP öncelikli, ancak gerekçeli durumlarda değiştirilebilir" entegrasyon stratejisiyle uyumlu, istisnai bir B planıdır; Fastify ve Drizzle teknoloji kısıtları aynen korunacaktır.
İP2, İP9	Altyapı (OCI) Performans Yetersizliği: Projenin ana kısıtlarından biri olan Oracle Cloud Infrastructure (OCI) Ücretsiz Sürüm (4 OCPU, 24 GB RAM, ARM) kaynaklarının, özellikle AKIS Proto gibi karmaşık ajanların (İP7) eşzamanlı LLM çağrıları ve yoğun CPU/RAM kullanımı sırasında yetersiz kalması. PostgreSQL (R8) ve	1. Proaktif Mimari Optimizasyon: Bu risk en baştan (İP2) itibaren mimari kararlarla yönetilmektedir. Kaynak-yoğun Prisma ve Next.js yerine, bilinçli olarak çok düşük bellek/CPU ayak izine sahip Fastify ve Drizzle ORM seçilmiştir. 2. Erken Performans Testi: Prototip (İP6) aşamasından hemen önce, Ay 2'de yük testleri (load testing) uygulanarak sistemin darboğazları erkenden tespit edilecektir. 3. İş Yükü Sınırlaması (Kuyruk Mekanizması): Platform, kaynakların

	uygulamanın aynı VM'de çalışmasının yaratacağı kaynak rekabeti riski.	tükenmesini önlemek için aynı anda çalışan ajan sayısını (örn. en fazla 2) sınırlayacak bir kuyruk (queue) mekanizması içerecektir. 4. Veritabanı Ayarlaması (Tuning): PostgreSQL'in shared_buffers gibi bellek ayarları, 24GB RAM kısıtı içinde uygulama ile rekabet etmeyecek şekilde optimize edilecektir.
İP5, İP7	Yapay Zekâ Model Kalitesi ve Tutarsızlığı: Projenin maliyet kısıtı (sıfır bütçe) nedeniyle yalnızca OpenRouter üzerinden erişilen ücretsiz katmanlı LLM'lere (Llama 3.3, Gemini 2.0 Flash vb.) bağımlı olunması. Bu modellerin, özellikle AKIS Proto (İP7) için gereken karmaşık kod üretimi veya AKIS Trace (İP5) için test senaryosu çıkarma görevlerinde, yetersiz, tutarsız veya hatalı ("halüsinasyon") çıktılar üretme riski.	1. İleri Düzey İstem Mühendisliği (Prompt Engineering): Çıktı kalitesini artırmak için, tek seferlik basit istemler yerine, Metodoloji Bölüm 4.2.4'te açıklanan "Planla-Yürüt-Değerlendir" (Planner-Execute-Reflector) pipeline'ı gibi gelişmiş, çok adımlı istem zincirleri kullanılacaktır. 2. Model Esnekliği (OpenRouter): Platform, OpenRouter API'si sayesinde tek bir modele bağımlı değildir. Bir modelin (örn. Llama) yetersiz kaldığı görevde, ajan otomatik olarak başka bir modele (örn. Gemini) geçiş yapabilecek şekilde tasarlanacaktır. 3. Teknik Spike (Fizibilite): Geliştirmeye başlamadan önce, en riskli senaryo (İP7) için küçük, zaman sınırlı bir fizibilite (spike) çalışması yapılarak model kalitesi test edilecektir1.
Tümü	Kapsam Kayması (Scope Creep): Proje ilerledikçe, "olması güzel olur" (nice-to-have) niteliğinde, "Proje Kapsamı ve Gereksinimler" dokümanı dışında kalan yeni ve heyecan verici özelliklerin (örn. "bir de Kod İnceleme (Code Review) Ajanı ekleyelim") akademik takvimi tehlikeye atma pahasına plana dahil edilme baskısı2.	1. Katı Kapsam Yönetimi: Bu doküman (Proje Kapsamı), "tek doğru kaynak" olarak kabul edilecektir3. "Kapsam Dışı Olanlar" (Bölüm 2.3) listesi tüm taleplere karşı bir kalkan olarak kullanılacaktır. 2. WSJF Önceliklendirmesi: Tüm yeni fikirler, metodolojide (Bölüm 4.5) belirtilen WSJF (Ağırlıklı En Kısa İşe Öncelik) formülüne göre objektif bir şekilde değerlendirilecektir. 3. Fikir Dondurma (Park Etme): Yeni fikirler reddedilmeyecek, ancak projenin bu fazı için "dondurulacak" ve "Gelecek Sürüm / Faz-3" iş listesine 4 eklenerek değerlendirmesi ertelenecektir.
Tümü	Teknik Borç Yığılması: Akademik takvim baskısı (özellikle İP6 ve İP10 demo teslimleri) nedeniyle, "şimdilik çalışsın" mantığıyla aceleye	1. "Bitti" Tanımı (DoD) Zorunluluğu: Metodolojide (Bölüm 4.6.2) tanımlanan "Definition of Done" (DoD) kontrol listesi (testlerin geçmesi, dokümantasyonun güncellenmesi vb.) her görev için zorunlu tutulacaktır.

	getirilen kodlar, yazılmayan testler veya ertelenen yeniden yapılandırma (refactoring) çalışmaları. Bu durumun, projenin ilerleyen aşamalarında (örn. İP7) geliştirmeyi yavaşlatması ve hatalara (bug) neden olması riski.	2. Kod İncelemesi (Code Review): İki kişilik ekip (Ömer, Ayşe), birbirlerinin kodlarını ana branch'e birleştirmeden önce mutlaka gözden geçirecektir (Code Review). 3. CI/CD Kalite Kapıları: GitHub Actions üzerinde kurulacak CI pipeline'ı (Bölüm 4.6.3), lint kurallarına uymayan veya testleri geçemeyen kodların birleştirilmesini otomatik olarak engelleyecektir.
İP1, İP10	Akademik Raporlama Yükü: Projenin kodlama eforu (İP2, İP5, İP7) ile akademik raporlama (İP1, İP10) eforu arasında dengesizlik yaşanması. Kodlamaya odaklanıp dokümantasyonu son haftaya bırakmanın, son teslim (Mayıs 22) tarihinde yetersiz veya aceleye gelmiş bir akademik rapor teslim etme riski.	1. Paralel Dokümantasyon (Dual-Track Agile): Metodolojide (Bölüm 4.4.1) belirtildiği gibi, proje "Dual-Track Agile" (Çift Hatlı Çevik) metodolojisini benimser. Geliştirme (Delivery) hattı ilerlerken, Dokümantasyon (Discovery/Research) hattı da paralel olarak yürütülecek ve rapor bölümleri (örn. Metodoloji, Tasarım) tamamlanan işlerle birlikte anlık olarak yazılacaktır. 2. Sprint'lere Dokümantasyon Görevleri Ekleme: Her sprint planlamasına, o sprintte yapılan işle ilgili akademik rapor bölümünü yazmak için özel görevler (task) eklenecektir.